

INFORME DE PROYECTO ECOMMERCE

PROGRAMACIÓN ORIENTADA A OBJETOS 2

Integrantes del grupo:

- Nahuel Isabella Valenzi | valenzinahuel@gmail.com
- Agustín Brizuela | agusbrizu.10@gmail.com
- Luca Moretti | lucamorettiar2005@gmail.com

Decisiones de diseño, detalles de implementación y patrones utilizados

2.1 Catalogo de productos:

Para resolver este problema decidimos implementar el catalogo de productos como una estructura jerárquica del patrón **Composite**.

Se definió la clase **CatalogoDeProductos** como la clase abstracta que toma el papel de componente según el libro de Gamma.

Producto actúa como la hoja (leaf) y Paquete actúa como compuesto (Composite), manteniendo una lista de **CatalogoDeProductos** lo que permite resolver el problema de tratar a ambos hoja y compuesto como uno mismo.

Para permitir atributos dinámicos en producto hicimos que producto tenga un atributo que sea un HashMap el cual tenga dentro atributos con su nombre como key y como value su valor.

2.2 Ciclo de vida de pedido:

Para resolver este problema modelamos el patrón **State** para cambiar su estado sin necesidad de un if o un switch.

Cada estado es una clase que implementa la interfaz **EstadoDePedido** la cual tiene el papel de Estado según el libro de Gamma y cada estado (**Borrador**, **Confirmado**, **EnPreparación**, **Enviado**, **Entregado**, **Cancelado**) sería un Estado Concreto. Y, por último, **Pedido** sería el Contexto en el patrón.

Cada estado decide si una operación es valida en ese contexto, si no lo es lanza una excepción de tipo **OperacionInvalidaException**.

2.3 Método de envío:

Para modelar los métodos de envío decidimos implementar el patron **Strategy** para poder elegir el método de forma dinámica en tiempo de ejecución.

La interfaz **MetodoDeEnvio** es la Estrategia y sus subclases **EnvioEstandar**, **EnvioExpress** y **RetiroEnSucursal** serían las Estrategias Concretas, siendo Pedido el Contexto.

Una decisión que tomamos para las librerías externas dadas por la consigna fue modelarlas como interfaces. Usamos el mismo criterio para las API de método de pago en la consigna lo que permite sustituir las librerías por un mock en los test.

Según la consigna se nos daba “Float EnvioExpress calcularCosto(Float precio);” por lo cual asumimos que se estaba refiriendo a una librería externa y no a la clase **EnvioExpress** que debíamos crear.

2.4 Métodos de pago:

Como realizar un pago sigue la misma secuencia de cuatro pasos (validar datos, reservar fondos, ejecutar transacción y notificar resultado) pero cada medio de pago lo hace de forma distinta decidimos aplicar el patrón **Template Method**.

La clase abstracta es **MetodosDePago**, la cual tiene el método concreto finalizarPago(Pedido pedido), que va ser el método plantilla para las subclases: **TarjetaDeCredito**, **TransferenciaBancaria** y **BilleteraVirtual**.

Cada método de pago opera con una interfaz propia que representa la API (**APITarjetaDeCredito**, **APITransferenciaBancaria**, **APIBilleteraVirtual**). Como indicaba la consigna las interfaces se definieron, pero no se implementaron.

Cada error lanza un **MetodoDePagoException** con un mensaje personalizado el cual describe el motivo del fallo.

2.5 Notificaciones del Pedido:

Cada vez que un pedido cambiaba de estado distintos subsistemas debían reaccionar de forma independiente, por lo que esto se resolvió con el patrón **Observer**.

Pedido actúa como Sujeto según el libro de Gamma, tiene una lista de observadores a los cuales notifica cuando cambia de estado. El Observador es la

interfaz **ObservadorDePedido**, los ObservadoresConcretos son:
NotificadorDeMail, **GeneradorDeFactura** y **Fidelizacion**

Tanto como **NotificadorDeMail** y **Fidelizacion** utilizan al actualizar la librería **MailSender** la cual representamos como una interfaz.

2.6 Búsqueda en el catálogo:

Para modelar los criterios de búsqueda usamos de nuevo el patrón **Composite**. **CriterioSimple** y **CriterioComplejo** son clases abstractas, y sus subclasses implementan la lógica concreta de cada criterio.

CriterioSimple agrupa a los criterios "hoja" (**CriterioPorNombre**, **CriterioPorPrecioMaximo**, **CriterioPorCategoria**, **CriterioPorStock**), que evalúan una condición puntual sobre un ítem. **CriterioComplejo** agrupa a los operadores lógicos (**CriterioAND**, **CriterioOR**, **CriterioNOT**), que combinan otros criterios (simples o complejos).

Con esto, el buscador no necesita distinguir si recibió un criterio simple o uno compuesto: para él, todos son un Criterio.

2.8 Reportes:

Para los reportes usamos el patrón **Visitor**. **Producto** y **Paquete** implementan **ReporteVisitable**, que tiene un método aceptar(**ReporteVisitor**) que delega en el visitor recibido.

ReporteDeProductosMasVendidos es el visitor concreto que recorre el catálogo visitando cada **Producto** y **Paquete**, y arma la lista **productosMasVendidos** ordenada según lo vendido.

Para la exportación a distintos formatos usamos la interfaz **ExportadorReporte** que define exportar(List<CatalogoDeProductos>): String, e implementaciones como **ReporteTextoPlano**, **ReporteCSV** y **ReporteHTML** generan el string en cada formato. Así, **ReporteDeProductosMasVendidos** se encarga de qué datos recolectar, y el exportador de cómo presentarlos.